

5D Projected Entity

Mathematics & Implementation

missvector
github.com/vifirsanova

May 20, 2026

Contents

1	Overview	3
2	Parameter Initialization	3
2.1	Frequencies	3
2.2	Phases	3
2.3	Modulators	3
2.4	Code	4
3	5D Trajectory	4
3.1	Formula	4
3.2	Code	4
4	Triple Perspective Projection	5
4.1	Step 1: $\mathbb{R}^5 \rightarrow \mathbb{R}^4$	5
4.2	Step 2: $\mathbb{R}^4 \rightarrow \mathbb{R}^3$	5
4.3	Step 3: $\mathbb{R}^3 \rightarrow \mathbb{R}^2$	5
4.4	Code	5
5	Time and Speed	6
5.1	Speed Presets	6
5.2	Code	6
6	Input Processing	7
6.1	Cursor	7
6.2	Scroll	7
6.3	Code	7
7	Trail Buffer	8
7.1	Storage	8
7.2	Update	8
7.3	Rendering	8
7.4	Code	8
8	Node Rendering	9
8.1	Size from Depth	9
8.2	Glow Cache	9
8.3	Code	9

9	Background Fade	10
10	Edge Drawing	11
11	Phase Perturbation on Click	11
12	Canvas Setup and Main Loop	12
12.1	Coordinate Transform	12
12.2	Code	12
13	Summary of Constants	13

1 Overview

The system defines a graph of $N = 56$ nodes, each moving through a 5-dimensional trajectory determined by coupled trigonometric oscillators. Three successive perspective projections map $\mathbb{R}^5 \rightarrow \mathbb{R}^4 \rightarrow \mathbb{R}^3 \rightarrow \mathbb{R}^2$. User input (mouse position, scroll, click) modifies projection parameters and time scaling.

- **Nodes:** $i \in \{0, 1, \dots, 55\}$
- **Edges:** Each node i connects to $(i + j) \bmod 56$ for $j \in \{1, \dots, 6\}$
- **Trail buffer:** $K = 50$ previous 2D positions stored per node
- **Glow cache:** 16 pre-rendered radial gradient canvases (radii 1–16 px)

2 Parameter Initialization

2.1 Frequencies

For dimension $d \in \{0, 1, 2, 3, 4\}$:

$$f_{i,d} = b_d + (i \bmod k_d) \cdot s_d \quad (1)$$

d	b_d	k_d	s_d
0	0.70	7	0.12
1	0.60	5	0.14
2	0.80	3	0.16
3	0.55	4	0.18
4	0.65	6	0.11

2.2 Phases

All values taken modulo 2π :

$$\begin{aligned} \varphi_{i,0} &= i \cdot 1.618034 \cdot 0.7 \\ \varphi_{i,1} &= i \cdot 0.87 + i^2 \cdot 0.031 \\ \varphi_{i,2} &= i \cdot 1.23 + i \cdot 0.57 \\ \varphi_{i,3} &= i \cdot 0.53 + i \cdot 0.91 \\ \varphi_{i,4} &= i \cdot 1.67 + i \cdot 0.23 \end{aligned}$$

2.3 Modulators

$$m_{i,d} = b'_d + (i \cdot s'_d) \bmod r_d \quad (2)$$

d	b'_d	s'_d	r_d
0	0.40	0.073	0.60
1	0.35	0.091	0.55
2	0.45	0.067	0.50
3	0.50	0.083	0.45
4	0.38	0.077	0.58

2.4 Code

Listing 1: Parameter initialization

```
1 const N = 56;
2 const phi = 1.618033988749895;
3
4 const freqs = new Float32Array(N * 5);
5 const phases = new Float32Array(N * 5);
6 const mods = new Float32Array(N * 5);
7
8 for (let i = 0; i < N; i++) {
9   const base = i * 5;
10  freqs[base] = 0.7 + (i % 7) * 0.12;
11  freqs[base+1] = 0.6 + (i % 5) * 0.14;
12  freqs[base+2] = 0.8 + (i % 3) * 0.16;
13  freqs[base+3] = 0.55 + (i % 4) * 0.18;
14  freqs[base+4] = 0.65 + (i % 6) * 0.11;
15
16  phases[base] = (i * phi * 0.7) % (Math.PI * 2);
17  phases[base+1] = (i * 0.87 + i*i * 0.031) % (Math.PI * 2);
18  phases[base+2] = (i * 1.23 + i * 0.57) % (Math.PI * 2);
19  phases[base+3] = (i * 0.53 + i * 0.91) % (Math.PI * 2);
20  phases[base+4] = (i * 1.67 + i * 0.23) % (Math.PI * 2);
21
22  mods[base] = 0.4 + (i * 0.073) % 0.6;
23  mods[base+1] = 0.35 + (i * 0.091) % 0.55;
24  mods[base+2] = 0.45 + (i * 0.067) % 0.5;
25  mods[base+3] = 0.5 + (i * 0.083) % 0.45;
26  mods[base+4] = 0.38 + (i * 0.077) % 0.58;
27 }
```

3 5D Trajectory

3.1 Formula

For node i , dimension $d \in \{0, 1, 2, 3, 4\}$, at time t :

$$c_{i,d}(t) = 0.55 \sin \alpha_1 + 0.28 \cos \alpha_2 + 0.20 \sin \alpha_3 + 0.13 \cos \alpha_4 \sin \alpha_5 \quad (3)$$

where:

$$\begin{aligned} \alpha_1 &= t \cdot f_{i,d} + \varphi_{i,d} \\ \alpha_2 &= t \cdot f_{i,d} \cdot 1.7 + m_{i,d} \cdot 3 \\ \alpha_3 &= t \cdot f_{i,(d+2) \bmod 5} \cdot 2.3 + i \cdot 0.13 \\ \alpha_4 &= t \cdot 0.43 + i \cdot 1.7 + d \\ \alpha_5 &= t \cdot 0.97 + i \cdot 0.5 \end{aligned}$$

The full 5D position vector:

$$\mathbf{c}_i(t) = (c_{i,0}(t), c_{i,1}(t), c_{i,2}(t), c_{i,3}(t), c_{i,4}(t)) \quad (4)$$

3.2 Code

Listing 2: 5D coordinate computation

```
1 const c5 = new Float32Array(5); // reusable scratch array
```

```

2
3 function compute5DCoords(i, t) {
4   const base = i * 5;
5
6   for (let dim = 0; dim < 5; dim++) {
7     const f = freqs[base + dim];
8     const p = phases[base + dim];
9     const m = mods[base + dim];
10    const f2 = freqs[base + (dim + 2) % 5];
11
12    let v = Math.sin(t * f + p) * 0.55;
13    v += Math.cos(t * f * 1.7 + m * 3) * 0.28;
14    v += Math.sin(t * f2 * 2.3 + i * 0.13) * 0.2;
15    v += Math.cos(t * 0.43 + i * 1.7 + dim)
16      * Math.sin(t * 0.97 + i * 0.5) * 0.13;
17
18    c5[dim] = v;
19  }
20 }

```

4 Triple Perspective Projection

4.1 Step 1: $\mathbb{R}^5 \rightarrow \mathbb{R}^4$

$$\begin{aligned}
 x'_4 &= x_4 + S \\
 w_4 &= 2.8 + x'_4 \\
 y_j &= \frac{2.8 \cdot x_j}{w_4}, \quad j \in \{0, 1, 2, 3\}
 \end{aligned} \tag{5}$$

4.2 Step 2: $\mathbb{R}^4 \rightarrow \mathbb{R}^3$

$$\begin{aligned}
 w_3 &= 2.8 + y_3 + V + 0.5 \cdot S \\
 z_0 &= \frac{2.8 \cdot y_0}{w_3} + 0.35 \cdot H \\
 z_1 &= \frac{2.8 \cdot y_1}{w_3} \\
 z_2 &= \frac{2.8 \cdot y_2}{w_3}
 \end{aligned} \tag{6}$$

4.3 Step 3: $\mathbb{R}^3 \rightarrow \mathbb{R}^2$

$$\begin{aligned}
 w_2 &= 3.5 + z_2 \\
 x_{\text{screen}} &= \frac{2.5 \cdot z_0}{w_2} \\
 y_{\text{screen}} &= \frac{2.5 \cdot z_1}{w_2} \\
 z_{\text{depth}} &= z_2
 \end{aligned} \tag{7}$$

4.4 Code

Listing 3: Projection function

```

1 function project5Dto2D(w40ff, v40ff, scrollZ) {
2   const x0 = c5[0], x1 = c5[1], x2 = c5[2], x3 = c5[3], x4 = c5[4];
3
4   // Step 1: 5D -> 4D
5   const x4s = x4 + scrollZ;
6   const iw4 = 1.0 / (2.8 + x4s);
7   const y0 = x0 * iw4 * 2.8;
8   const y1 = x1 * iw4 * 2.8;
9   const y2 = x2 * iw4 * 2.8;
10  const y3 = x3 * iw4 * 2.8;
11
12  // Step 2: 4D -> 3D
13  const iw3 = 1.0 / (2.8 + y3 + v40ff + scrollZ * 0.5);
14  const z0 = y0 * iw3 * 2.8 + w40ff * 0.35;
15  const z1 = y1 * iw3 * 2.8;
16  const z2 = y2 * iw3 * 2.8;
17
18  // Step 3: 3D -> 2D
19  const iw2 = 1.0 / (3.5 + z2);
20  return {
21    x: z0 * iw2 * 2.5, // x_screen
22    y: z1 * iw2 * 2.5, // y_screen
23    z: z2              // z_depth
24  };
25 }

```

5 Time and Speed

5.1 Speed Presets

$$\sigma \in \Sigma = \{0.05, 0.15, 0.35, 0.60, 1.00, 1.80, 3.20, 5.50\} \quad (8)$$

On click, index cycles modulo 8:

$$\sigma_{\text{target}} = \Sigma[(\text{index} + 1) \bmod 8] \quad (9)$$

Per frame, exponential smoothing (lerp factor $\lambda_\sigma = 0.08$):

$$\sigma \leftarrow \sigma + (\sigma_{\text{target}} - \sigma) \cdot \lambda_\sigma \quad (10)$$

Simulation time:

$$t = t_{\text{wall}} \cdot \sigma \quad (11)$$

5.2 Code

Listing 4: Speed control

```

1 const SPEED_PRESETS = [0.15, 0.35, 0.6, 1.0, 1.8, 3.2, 5.5, 0.05];
2 let speedIndex = 3;
3 let currentSpeed = SPEED_PRESETS[speedIndex];
4 let targetSpeed = currentSpeed;
5
6 canvas.addEventListener('click', () => {
7   speedIndex = (speedIndex + 1) % SPEED_PRESETS.length;
8   targetSpeed = SPEED_PRESETS[speedIndex];
9   // phase perturbation happens here (see Section 9)
10 });
11
12 // per frame:

```

```

13 currentSpeed += (targetSpeed - currentSpeed) * 0.08;
14 const t = timestamp * 0.001 * currentSpeed;

```

6 Input Processing

6.1 Cursor

Normalized mouse coordinates relative to canvas center:

$$\hat{x}_{\text{mouse}}, \hat{y}_{\text{mouse}} \in [-0.5, 0.5] \quad (12)$$

Exponential smoothing ($\lambda_m = 0.05$):

$$m_x \leftarrow m_x + (\hat{x}_{\text{mouse}} - m_x) \cdot \lambda_m \quad (13)$$

$$m_y \leftarrow m_y + (\hat{y}_{\text{mouse}} - m_y) \cdot \lambda_m \quad (14)$$

Projection inputs:

$$H = m_x \cdot 1.17, \quad V = m_y \cdot 1.17 \quad (15)$$

6.2 Scroll

Accumulation:

$$S_{\text{target}} \leftarrow S_{\text{target}} + \Delta_{\text{wheel}} \cdot 0.0008 \quad (16)$$

$$S_{\text{target}} = \text{clamp}(S_{\text{target}}, -3, 3) \quad (17)$$

Smoothing ($\lambda_s = 0.06$):

$$S \leftarrow S + (S_{\text{target}} - S) \cdot \lambda_s \quad (18)$$

6.3 Code

Listing 5: Input handling

```

1 let mx = 0, my = 0, tmx = 0, tmy = 0;
2
3 canvas.addEventListener('mousemove', e => {
4   const r = canvas.getBoundingClientRect();
5   tmx = (e.clientX - r.left) / W - 0.5;
6   tmy = (e.clientY - r.top) / H - 0.5;
7 });
8 canvas.addEventListener('mouseleave', () => { tmx = 0; tmy = 0; });
9
10 let scrollOffset = 0, targetScroll = 0;
11
12 window.addEventListener('wheel', (e) => {
13   e.preventDefault();
14   targetScroll += e.deltaY * 0.0008;
15   targetScroll = Math.max(-3, Math.min(3, targetScroll));
16 }, { passive: false });
17
18 // per frame:
19 mx += (tmx - mx) * 0.05;
20 my += (tmy - my) * 0.05;
21 scrollOffset += (targetScroll - scrollOffset) * 0.06;
22
23 const w40ff = mx * 1.17;
24 const v40ff = my * 1.17;

```

7 Trail Buffer

7.1 Storage

Ring buffer of $K = 50$ 2D positions per node:

$$\mathbf{p}_{i,k} \in \mathbb{R}^2, \quad k \in \{0, 1, \dots, K-1\} \quad (19)$$

$k = 0$ is the current frame, $k = K-1$ is the oldest.

7.2 Update

Per frame, shift backward:

$$\mathbf{p}_{i,k} \leftarrow \mathbf{p}_{i,k-1}, \quad k = K-1, \dots, 1 \quad (20)$$

Write new position:

$$\mathbf{p}_{i,0} = (x_{\text{screen},i}(t), y_{\text{screen},i}(t)) \quad (21)$$

7.3 Rendering

Segments drawn from $\mathbf{p}_{i,k}$ to $\mathbf{p}_{i,k-1}$ with:

$$\alpha_k = \left(1 - \frac{k}{K}\right) \cdot 0.5 \cdot q_i(t) \quad (22)$$

$$w_k = 0.8 \cdot q_i(t) \cdot \left(1 - \frac{k}{K}\right) \quad (23)$$

where the **pulse** is:

$$q_i(t) = 0.6 + \sin(t \cdot 5 + \varphi_{i,0}) \cdot 0.4 \quad (24)$$

7.4 Code

Listing 6: Trail update and drawing

```
1 const TRAIL_LENGTH = 50;
2 const trailX = new Float32Array(N * TRAIL_LENGTH);
3 const trailY = new Float32Array(N * TRAIL_LENGTH);
4
5 // Shift (per frame)
6 for (let i = 0; i < N; i++) {
7   const base = i * TRAIL_LENGTH;
8   for (let k = TRAIL_LENGTH - 1; k > 0; k--) {
9     trailX[base + k] = trailX[base + k - 1];
10    trailY[base + k] = trailY[base + k - 1];
11  }
12 }
13
14 // Write new
15 for (let i = 0; i < N; i++) {
16   compute5DCoords(i, t);
17   const p = project5Dto2D(w40ff, v40ff, scroll10ffset);
18   trailX[i * TRAIL_LENGTH] = p.x;
19   trailY[i * TRAIL_LENGTH] = p.y;
20 }
21
22 // Draw
23 for (let i = 0; i < N; i++) {
24   const base = i * TRAIL_LENGTH;
```

```

25   const pulse = 0.6 + Math.sin(t * 5 + phases[i * 5]) * 0.4;
26
27   for (let k = TRAIL_LENGTH - 1; k >= 1; k--) {
28     const alpha = (1 - k / TRAIL_LENGTH) * 0.5 * pulse;
29     if (alpha < 0.01) continue;
30
31     const x1 = cx + trailX[base + k] * sc;
32     const y1 = cy + trailY[base + k] * sc;
33     const x2 = cx + trailX[base + k - 1] * sc;
34     const y2 = cy + trailY[base + k - 1] * sc;
35
36     ctx.strokeStyle = `rgba(255,255,255,${alpha})`;
37     ctx.lineWidth = 0.8 * pulse * (1 - k / TRAIL_LENGTH);
38     ctx.beginPath();
39     ctx.moveTo(x1, y1);
40     ctx.lineTo(x2, y2);
41     ctx.stroke();
42   }
43 }

```

8 Node Rendering

8.1 Size from Depth

Approximate depth from screen position:

$$\tilde{z}_i = 0.3 \cdot x_{\text{screen},i} + 0.3 \cdot y_{\text{screen},i} \quad (25)$$

Depth factor:

$$z_{\text{fact}} = (\tilde{z}_i + 1.5) \cdot 0.7 \quad (26)$$

Glow radius (clamped, rounded to integer for cache lookup):

$$r_{\text{glow}} = \text{clamp}(\lfloor 5 \cdot q_i(t) \cdot z_{\text{fact}} \rfloor, 1, 16) \quad (27)$$

Core radius:

$$r_{\text{core}} = \max(1.8 \cdot q_i(t) \cdot z_{\text{fact}}, 0.6) \quad (28)$$

8.2 Glow Cache

For $r \in \{1, \dots, 16\}$, pre-render a radial gradient on an offscreen canvas of size $(2r+6) \times (2r+6)$:

- Color stop 0 (center): `rgba(255,255,255,0.8)`
- Color stop 0.5: `rgba(255,255,255,0.2)`
- Color stop 1 (edge): `rgba(255,255,255,0)`

Stored as 16 canvas elements in `glowCanvases[r - 1]`.

8.3 Code

Listing 7: Node rendering

```

1  const glowCanvases = [];
2  const MAX_GLOW_SIZE = 16;
3
4  function buildGradients() {
5    glowCanvases.length = 0;

```

```

6   const off = document.createElement('canvas');
7   const offCtx = off.getContext('2d');
8
9   for (let r = 1; r <= MAX_GLOW_SIZE; r++) {
10    const d = r * 2 + 6;
11    off.width = d; off.height = d;
12    offCtx.clearRect(0, 0, d, d);
13
14    const half = d / 2;
15    const grd = offCtx.createRadialGradient(half, half, 0, half, half, r);
16    grd.addColorStop(0, 'rgba(255,255,255,0.8)');
17    grd.addColorStop(0.5, 'rgba(255,255,255,0.2)');
18    grd.addColorStop(1, 'rgba(255,255,255,0)');
19    offCtx.fillStyle = grd;
20    offCtx.beginPath();
21    offCtx.arc(half, half, r, 0, Math.PI * 2);
22    offCtx.fill();
23
24    const c = document.createElement('canvas');
25    c.width = d; c.height = d;
26    c.getContext('2d').drawImage(off, 0, 0);
27    glowCanvases.push(c);
28  }
29 }
30
31 // Drawing nodes (per frame)
32 for (let i = 0; i < N; i++) {
33   const x = cx + trailX[i * TRAIL_LENGTH] * sc;
34   const y = cy + trailY[i * TRAIL_LENGTH] * sc;
35
36   const approxZ = trailX[i * TRAIL_LENGTH] * 0.3
37                 + trailY[i * TRAIL_LENGTH] * 0.3;
38   const zFactor = (approxZ + 1.5) * 0.7;
39   const pulse = 0.6 + Math.sin(t * 5 + phases[i * 5]) * 0.4;
40   const glowR = Math.round(
41     Math.max(1, Math.min(16, 5 * pulse * zFactor))
42   );
43
44   // Glow from cache
45   const glowC = glowCanvases[glowR - 1];
46   if (glowC) {
47     ctx.drawImage(glowC,
48       Math.round(x - glowC.width / 2),
49       Math.round(y - glowC.height / 2)
50     );
51   }
52
53   // Core
54   const coreR = Math.max(0.6, 1.8 * pulse * zFactor);
55   ctx.fillStyle = 'ffffff';
56   ctx.beginPath();
57   ctx.arc(x, y, coreR, 0, Math.PI * 2);
58   ctx.fill();
59 }

```

9 Background Fade

$$\alpha_{bg} = \text{clamp}\left(\frac{0.25}{\sqrt{\sigma + 0.1}}, 0.08, 0.35\right) \quad (29)$$

Applied as: `rgba(10,10,10, $\alpha_{bg}$ )` over the full canvas.

At low σ , $\alpha_{\text{bg}} \rightarrow 0.35$ (more opaque clear, shorter visible trails). At high σ , $\alpha_{\text{bg}} \rightarrow 0.08$ (faint clear, long phosphorescent trails).

Listing 8: Background fade

```
1 const fadeAlpha = Math.max(0.08,
2   Math.min(0.35, 0.25 / Math.sqrt(currentSpeed + 0.1))
3 );
4 ctx.fillStyle = `rgba(10, 10, 10, ${fadeAlpha})`;
5 ctx.fillRect(0, 0, W, H);
```

10 Edge Drawing

Edge between node i and node j is drawn iff:

$$(x_i - x_j)^2 + (y_i - y_j)^2 < (0.7 \cdot L)^2 \quad (30)$$

where L is the canvas dimension in pixels, and $x_i = \text{cx} + \text{trailX}[i \cdot K] \cdot \text{sc}$.

Style: `rgba(255, 255, 255, 0.25)`, width 0.6px.

Listing 9: Edge drawing

```
1 ctx.strokeStyle = `rgba(255, 255, 255, 0.25)`;
2 ctx.lineWidth = 0.6;
3 ctx.beginPath();
4
5 for (let i = 0; i < N; i++) {
6   const x1 = cx + trailX[i * TRAIL_LENGTH] * sc;
7   const y1 = cy + trailY[i * TRAIL_LENGTH] * sc;
8
9   for (let j = 1; j <= 6; j++) {
10    const n = (i + j) % N;
11    const x2 = cx + trailX[n * TRAIL_LENGTH] * sc;
12    const y2 = cy + trailY[n * TRAIL_LENGTH] * sc;
13    const dx = x2 - x1;
14    const dy = y2 - y1;
15
16    if (dx * dx + dy * dy < size * size * 0.49) {
17      ctx.moveTo(x1, y1);
18      ctx.lineTo(x2, y2);
19    }
20  }
21 }
22 ctx.stroke();
```

11 Phase Perturbation on Click

When speed cycles, all phases receive additive uniform noise:

$$\varphi_{i,d} \leftarrow \varphi_{i,d} + \epsilon, \quad \epsilon \sim \text{Uniform}(-0.6, 0.6) \quad (31)$$

This causes the entity to shatter and reform in a new configuration.

Listing 10: Phase perturbation

```
1 canvas.addEventListener('click', () => {
2   speedIndex = (speedIndex + 1) % SPEED_PRESETS.length;
3   targetSpeed = SPEED_PRESETS[speedIndex];
4
5   for (let i = 0; i < N; i++) {
```

```

6   for (let dim = 0; dim < 5; dim++) {
7       phases[i * 5 + dim] += (Math.random() - 0.5) * 1.2;
8   }
9   }
10  });

```

12 Canvas Setup and Main Loop

12.1 Coordinate Transform

Canvas-to-world scaling:

$$\text{sc} = L \cdot 0.85 \quad (32)$$

Screen position from normalized coords:

$$x_{\text{px}} = \frac{L}{2} + x_{\text{screen}} \cdot \text{sc}, \quad y_{\text{px}} = \frac{L}{2} + y_{\text{screen}} \cdot \text{sc} \quad (33)$$

12.2 Code

Listing 11: Main loop structure

```

1  let W, H, size, cx, cy, sc;
2
3  function resize() {
4      size = Math.min(window.innerWidth * 0.95, window.innerHeight * 0.95, 700);
5      canvas.width = size;
6      canvas.height = size;
7      W = size; H = size;
8      cx = W / 2; cy = H / 2;
9      sc = size * 0.85;
10     buildGradients();
11 }
12 window.addEventListener('resize', resize);
13 resize();
14
15 function draw(timestamp) {
16     // Smooth inputs toward targets
17     currentSpeed += (targetSpeed - currentSpeed) * 0.08;
18     scrollOffset += (targetScroll - scrollOffset) * 0.06;
19     mx += (tmx - mx) * 0.05;
20     my += (tmy - my) * 0.05;
21
22     const t = timestamp * 0.001 * currentSpeed;
23     const w4Off = mx * 1.17;
24     const v4Off = my * 1.17;
25
26     // Shift trails, compute new positions
27     // ... (Sections 7-8)
28
29     // Background fade (Section 9)
30     // Draw edges (Section 10)
31     // Draw trails (Section 7)
32     // Draw nodes (Section 8)
33 }
34
35 function loop(ts) {
36     draw(ts);
37     requestAnimationFrame(loop);
38 }
39 requestAnimationFrame(loop);

```

13 Summary of Constants

Symbol	Value	Description
N	56	Number of nodes
K	50	Trail buffer length
L	$\min(vw \cdot 0.95, vh \cdot 0.95, 700)$	Canvas size (px)
Σ	$\{0.05, 0.15, 0.35, 0.60, 1.00, 1.80, 3.20, 5.50\}$	Speed presets
λ_σ	0.08	Speed lerp factor
λ_s	0.06	Scroll lerp factor
λ_m	0.05	Mouse lerp factor
Δ_{wheel}	0.0008	Scroll sensitivity
S_{max}	3	Scroll clamp range

Projection constant	Value	Used in
2.8	Base depth	w_4, w_3
3.5	Base depth	w_2
2.8	Scale factor	y_j, z_j numerators
2.5	Scale factor	$x_{\text{screen}}, y_{\text{screen}}$ numerators
0.35	Cursor X shift	z_0 offset
0.5	Scroll bleed	w_3